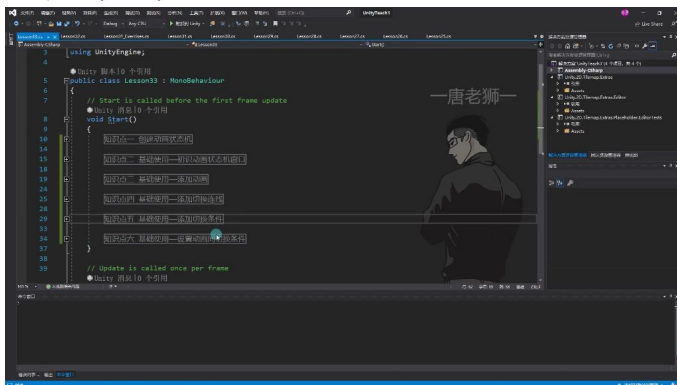
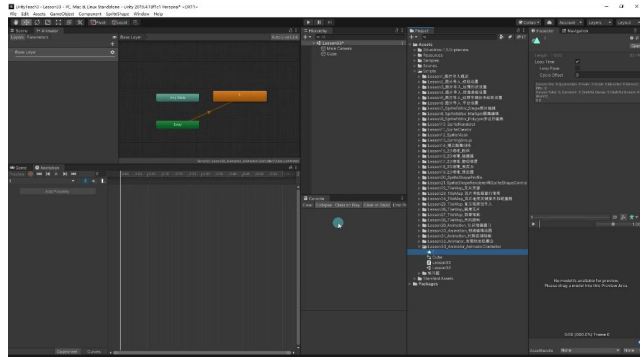


一、动画控制器状态机 00:04

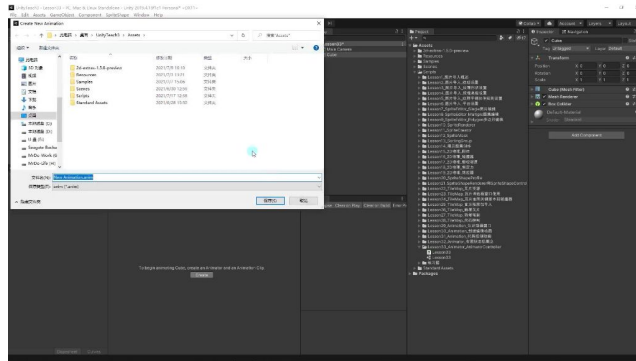
1. 创建动画状态机 00:42



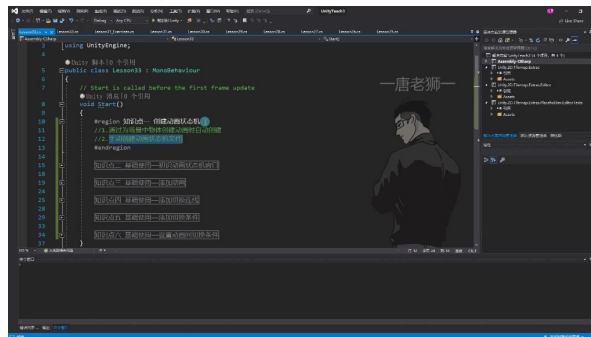
- 自动创建方式：
 - 当为场景物体创建动画时，Unity会自动完成以下操作：
 - 添加Animator组件
 - 创建以对象命名的状态机文件（如Cube.controller）
 - 创建动画文件并自动关联到状态机
 - 操作步骤：直接为物体创建动画 → 自动生成完整动画系统
- 手动创建方式：
 - 在Project窗口右键 → Create → Animator Controller
 - 特点：生成空状态机文件，需手动添加所有动画状态和过渡
 - 文件扩展名：.controller



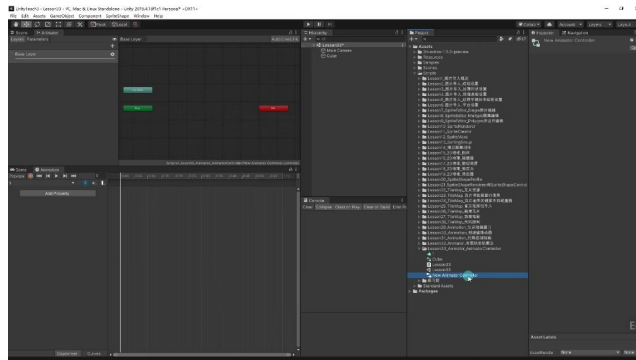
- 状态机差异：
 - 自动创建：包含初始动画状态（橙色矩形）
 - 手动创建：完全空白状态机窗口
- 组件关联：
 - 自动创建时会同时添加Animator组件并关联控制器
 - 手动创建后需手动将控制器拖拽到物体的Animator组件



- 演示准备：
 - 创建空场景
 - 添加立方体（位置设为0,0,0）
 - 两种创建方式的对比操作：
 - 方法一：通过Animation组件直接创建动画
 - 方法二：手动创建Animator Controller文件



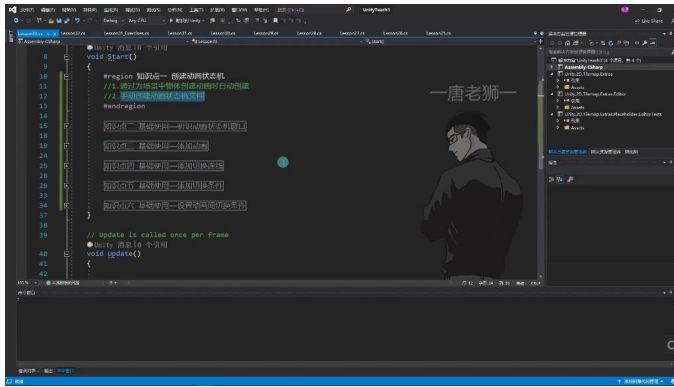
- 文件管理：
 - 建议在Assets下建立专用文件夹（如Lesson33）
 - 命名规范：建议使用"对象名_功能.controller"格式
 - 双击.controller文件可打开状态机编辑窗口



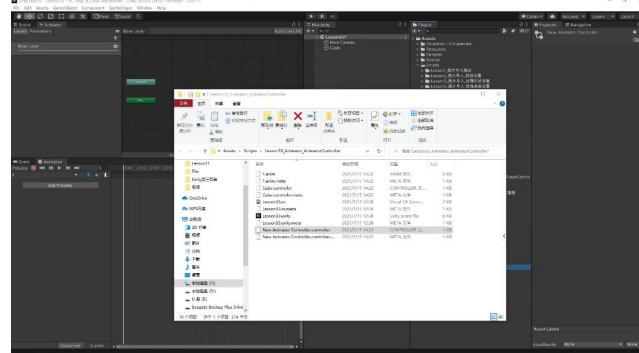
- 注意事项：
 - 老动画系统（Animation组件）不会自动创建状态机
 - 新动画系统（Animator）必须配合状态机使用
 - 自动创建的状态机默认包含Entry→默认动画的过渡

2. 基础使用 03:02

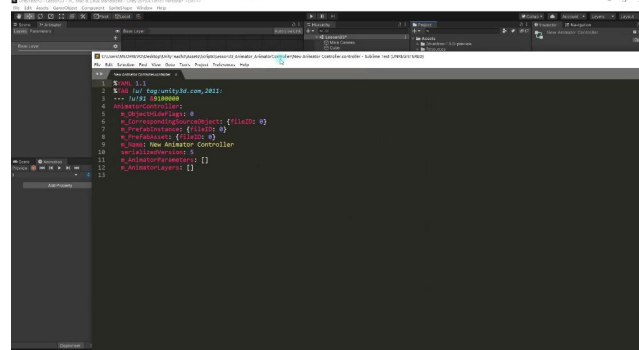
1) 初识动画状态机窗口



- 创建方式
 - 自动创建: 当为场景中的物体创建动画时, Unity会自动生成动画状态机文件
 - 手动创建: 可以在Assets目录下右键手动创建Animator Controller文件

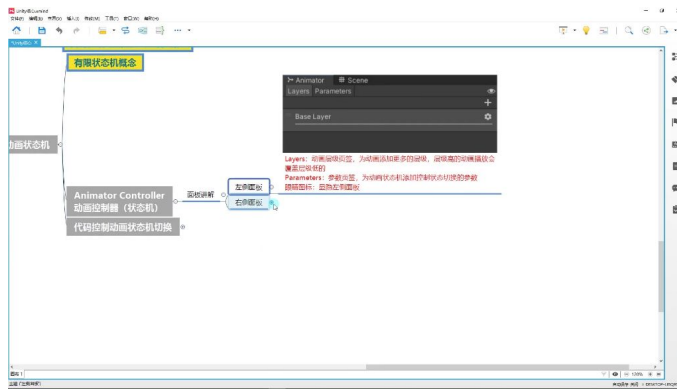


- 文件本质
 - 数据结构: 动画状态机文件本质是一系列存储动画参数和状态的数据集合
 - 文件格式: 使用YAML格式存储, 可通过文本编辑器查看原始数据内容

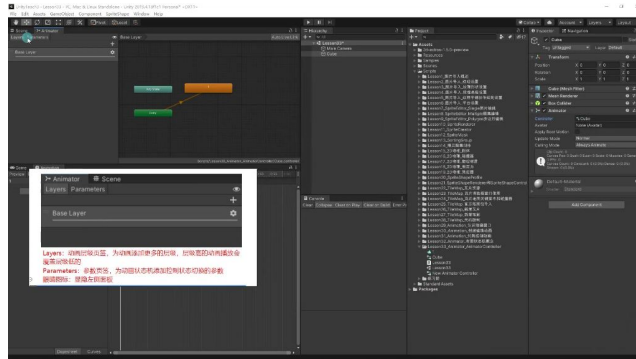


- 工作原理
 - 组件关联: 通过Animator组件关联状态机文件到游戏对象
 - 可视化编辑: Animator窗口提供图形界面编辑状态机数据
 - 数据存储: 所有动画参数和状态转换条件都存储在状态机文件中
 - 运行时控制: 通过代码调用状态机文件中存储的数据来控制动画播放
 - 工作流程:
 - 在Animator窗口可视化编辑状态机
 - 数据自动保存到状态机文件
 - 运行时通过代码调用状态机数据
 - 系统根据状态机逻辑播放对应动画

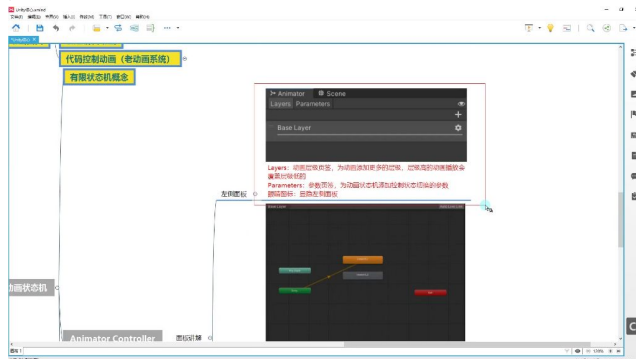
2) 动画控制器面板讲解 04:30



-
- 动画层级页签 05:04



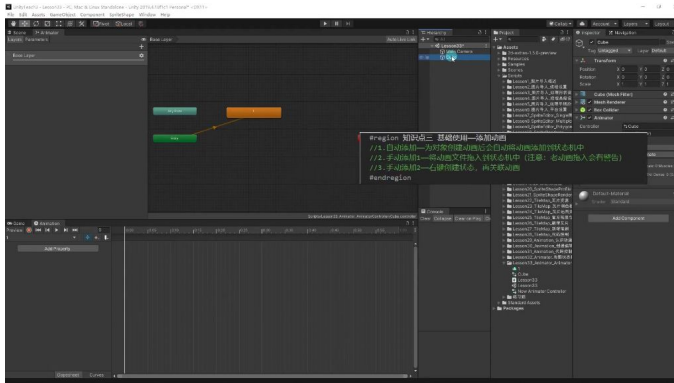
-
- 层级关系：通过"+"号可添加新层级，不同层级的动画可以同时播放
- 权重控制：层级齿轮设置中的权重值（0-1）决定是否覆盖其他层动画，权重为1时完全覆盖
- 操作方式：支持右键删除层级，层级名称可自定义修改
- 注意点：层级高低本身不决定覆盖关系，实际由权重参数控制
- 参数页签 06:20



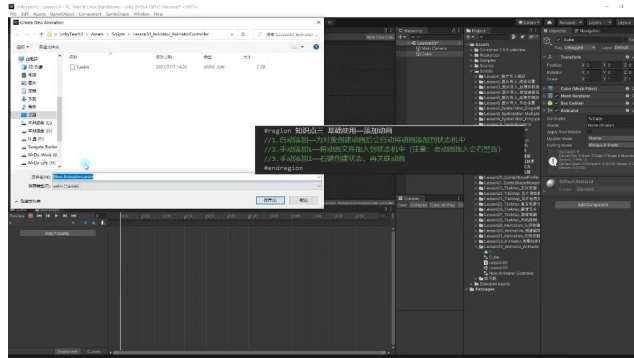
-
- 参数类型：提供Float、Int、Bool、Trigger四种条件参数
- 功能作用：用于控制动画状态之间的切换条件
- 添加方式：点击"+"号可选择创建不同类型的控制参数
- 应用场景：后续课程会详细讲解各参数的具体使用方法
- 网格化布局区域 06:56

- 可后续关联具体动画片段
- **连线操作:**
 - 可手动创建状态过渡
 - 选中连线后可在Inspector窗口添加切换条件
 - 按Delete键可删除不需要的连线

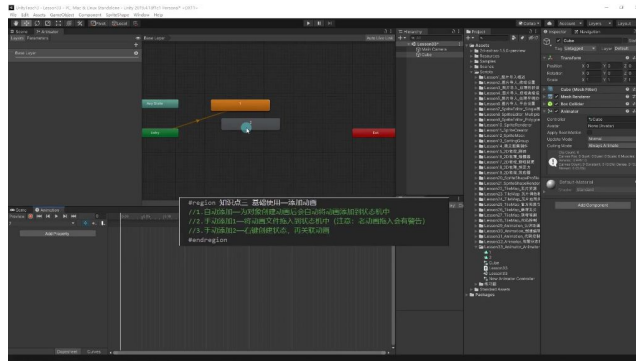
3) 基础使用—添加动画 10:01



- 自动添加方式
 - **创建原理:** 当为场景中的物体创建新动画时, Unity会自动将该动画添加到关联的状态机中
 - **操作示例:** 以Cube对象为例, 创建名为"动画二"的新动画文件后, 状态机窗口会自动显示该动画状态
 - **后续操作:** 自动添加后可直接编辑状态间的过渡关系, 如设置切换条件和连线
- 手动拖拽添加



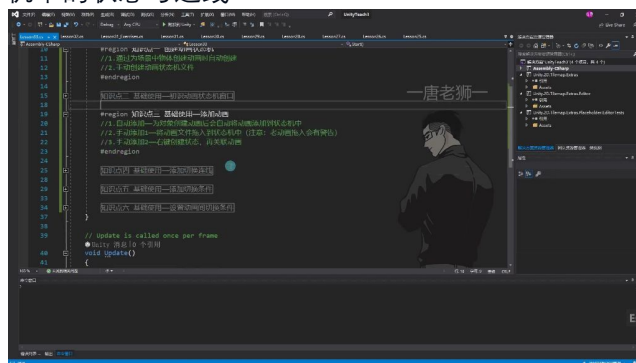
- 操作步骤:
 - 在Project窗口选中动画文件 (如"动画二")
 - 直接拖拽至Animator窗口
- 注意事项:
 - 老版动画系统文件会产生警告, 建议统一使用新版动画系统
 - 识别特征: 使用Animator组件而非旧版Animation组件
- 命名特性:
 - 状态名默认与动画文件名相同但可修改
 - 状态名与关联动画文件名可以不同 (通过Motion属性关联)
- 右键创建状态



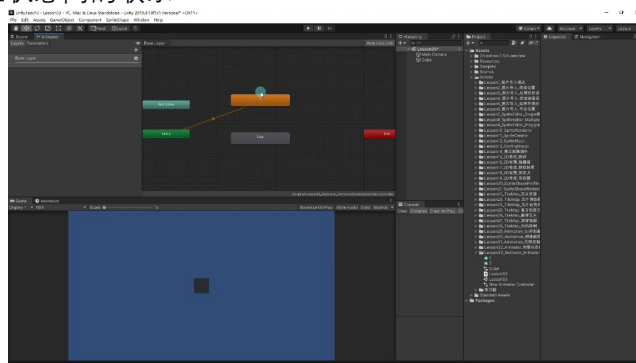
-
- 操作流程：
 - 在Animator窗口右键选择"Create State"→"Empty"
 - 命名新建状态（如"test"）
 - 将动画文件拖拽至该状态的Motion属性栏
- 状态特性：
 - 空状态初始不关联任何动画
 - 可随时更换关联的动画文件
 - 状态名应具有描述性以便识别（避免简单数字命名）
- 通用建议：
 - 优先使用自动添加方式确保一致性
 - 修改状态名时保持命名规范清晰
 - 复杂动画系统建议建立命名约定文档

4) 基础使用—添加切换连线 13:14

- 状态机中的状态与连线 13:21

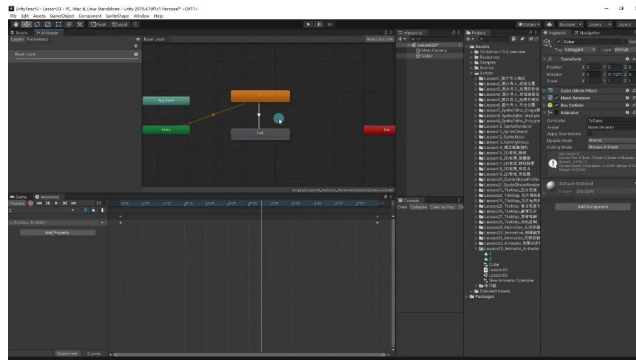


-
- 状态表示：矩形代表动画状态，如"Enter"默认状态和"Test"自定义状态
- 连线作用：箭头连线表示状态间的切换路径，建立单向联系
- 默认状态：橙色矩形表示默认播放的初始动画状态
- 建立状态间的联系 13:31

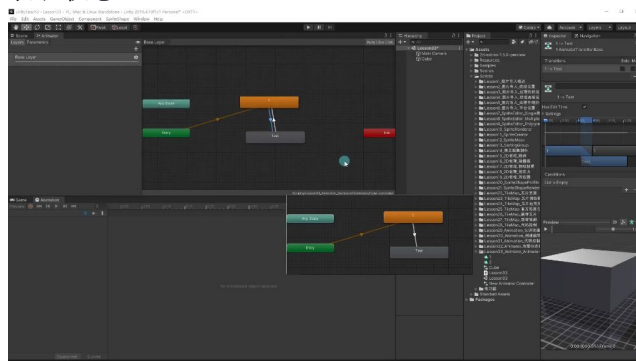


-
- 创建方法：
 - 右键选择源状态 → 选择"Make Transition" → 拖动到目标状态

- 连线方向决定切换方向（如状态A→状态B是单向切换）
- **连线验证**：运行时可观察蓝色进度条显示当前播放状态
- **循环切换**：双向连线可实现状态间的循环切换（如A↔B）
- **连线的作用与意义 14:16**

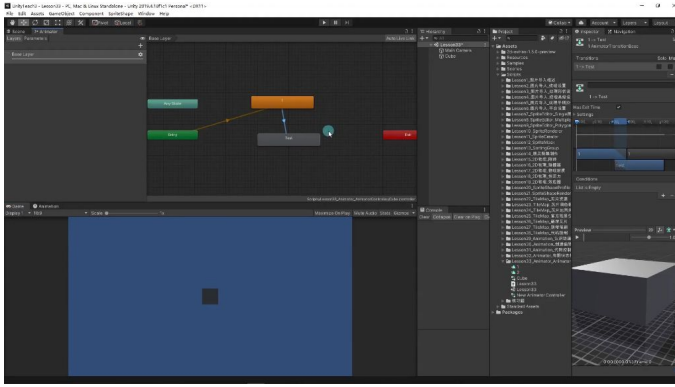


- **自动过渡**：未设置条件时，动画播放完毕自动触发切换
- **过渡效果**：系统会自动处理位置/旋转等属性的平滑过渡
- **中断播放**：删除连线后动画将在当前状态循环播放
- **实际案例**：
 - 状态1：Y轴位置移动10个单位（上升动画）
 - 状态2：Y轴旋转90度（旋转动画）
 - 通过连线实现"上升→旋转→上升"的循环效果
- **设置默认状态 18:02**

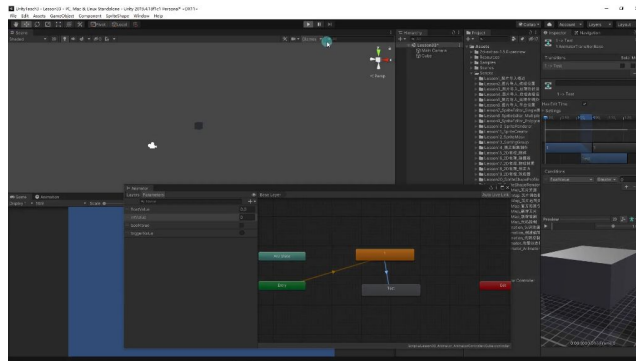


- **设置方法**：
 - 右键目标状态
 - 选择"Set as Layer Default State"
- **视觉标识**：默认状态显示为橙色矩形
- **应用场景**：需要改变初始播放动画时使用
- **注意事项**：
 - 新默认状态仍需保持必要连线
 - 未连线的默认状态将始终循环播放
 - 可通过Inspector窗口调整过渡参数

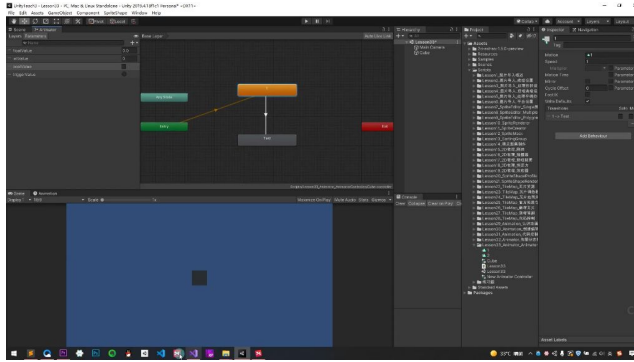
5) 基础使用-添加切换条件 19:09



-
- **参数类型：**支持四种变量类型（Float/Int/Bool/Trigger）
- **创建方法：**在Animator窗口左侧参数面板点击"+"号创建
- **命名规范：**建议使用value后缀（如floatValue）明确变量类型
- **四种类型的切换条件 19:22**

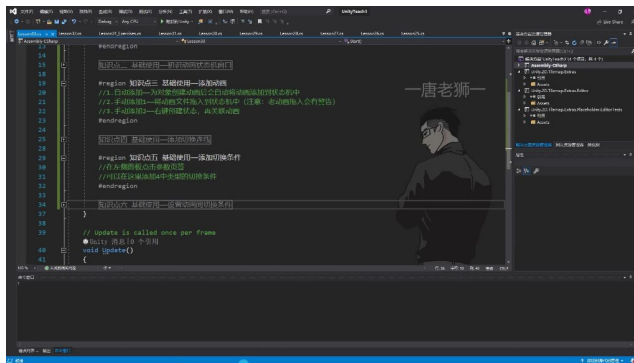


-
- **Float类型：**
 - 比较方式：仅支持>和<两种运算符
 - 应用场景：连续值控制（如速度阈值）
 - 示例：当floatValue > 0时触发状态切换
- **Int类型：**
 - 比较方式：支持>、<、=、!=四种运算符
 - 特殊用法：可通过等于判断实现精确状态切换
 - 示例：intValue = 2时切换到特定动画
- **Bool类型：**
 - 状态选项：仅true/false两种选择
 - 特点：需手动重置状态
 - 示例：isRunning为true时切换跑步动画
- **Trigger类型：**
 - 触发特性：单次触发后自动复位
 - 典型应用：一次性动作（如跳跃攻击）
 - 示例：jumpTrigger触发后立即复位
- **条件组合规则 23:12**



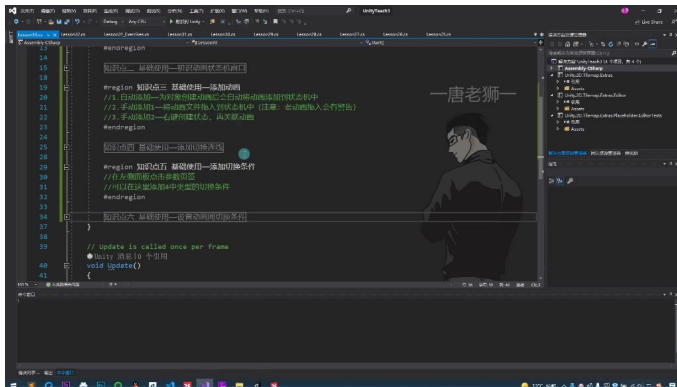
- 逻辑关系：多个条件间为"与"关系，需同时满足
- 优先级：无优先级区分，按列表顺序判断
- 示例组合：
 - floatValue < 0
 - intValue > 1
 - 需同时满足才会触发切换

● Trigger的特殊性质 24:34



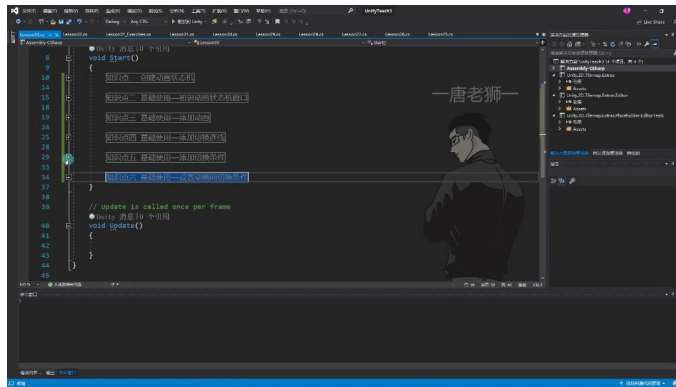
- 自动复位：触发后立即恢复未触发状态
- 连线建议：适合双向状态切换（如站立→跳跃→站立）
- 代码控制：通过SetTrigger()方法触发

6) 基础使用-设置动画间切换条件 26:34



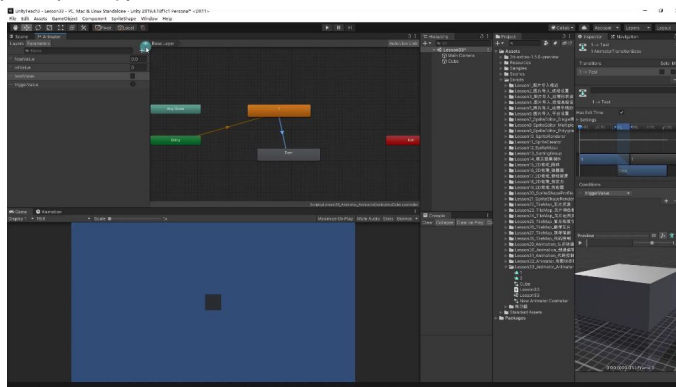
- 操作步骤：
 - 选择状态转换箭头
 - 在Inspector窗口添加条件
 - 设置参数判断规则
- 最佳实践：
 - 简单动画：使用Bool类型
 - 精确控制：使用Int等于判断
 - 物理模拟：使用Float阈值判断
 - 瞬时动作：使用Trigger类型

3. Unity动画状态机基础 27:05



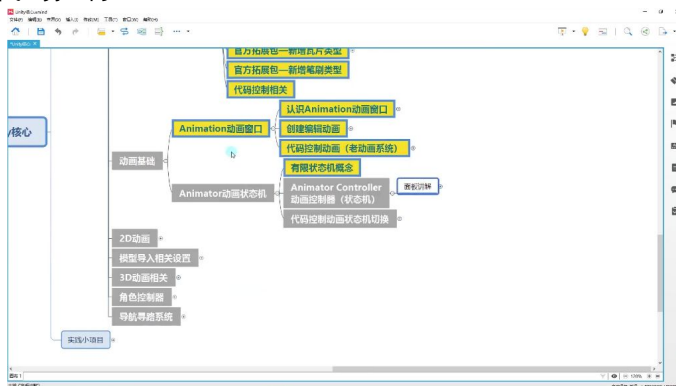
- 创建方法：通过Assets菜单新建Animator Controller
- 核心组件：
 - States：包含Idle/Run/Jump等动画状态
 - Transitions：状态间的连线箭头
 - Parameters：控制切换的变量（Bool/Trigger等）

1) 状态机操作



- 添加状态：右键空白处→Create State→From New Blend Tree
- 连线技巧：
 - 右键源状态→Make Transition→选择目标状态
 - 拖动连线可调整弧度
- 条件设置：
 - 选中连线→Conditions栏添加判断条件
 - 典型条件：Speed参数>0.5时切换到Run状态

2) 代码控制



- 核心API：
 - Animator.SetBool("参数名",值)
 - Animator.SetTrigger("参数名")

- **执行流程：**
 - 代码修改Parameters值
 - 状态机检测条件满足
 - 自动执行状态切换
 - **注意事项：**
 - Trigger类型参数会自动复位
 - 状态切换时可设置过渡时间(Settings→Transition Duration)
- 3) 应用实例
- **典型场景：**
 - 角色移动：Idle↔Run状态切换
 - 战斗系统：Attack→Hit→Death状态链
 - **调试技巧：**
 - 运行时观察状态机当前激活状态
 - 使用Debug.Log输出参数变化

记忆要点：

二、知识小结

知识点	核心内容	操作要点	难度系数
创建动画状态机	自动创建与手动创建方式	通过Animation窗口自动生成/右键Create Animator Controller	★★☆☆☆
状态机窗口结构	左侧参数面板与右侧状态编辑区	层级管理/参数类型(float,int,bool,trigger)/状态连线	★★★☆☆
添加动画	三种添加方式： 1. 自动添加 2. 拖拽添加 3. 右键创建空状态后关联	注意老动画系统兼容性问题	★★☆☆☆
状态切换连线	建立状态间转移关系	右键Make Transition生成箭头连线	★★★☆☆
切换条件设置	四种参数类型控制： - Float：大于/小于 - Int：大于/小于/等于/不等于 - Bool：true/false - Trigger：一次性触发	多条件为AND关系	★★★★☆
默认状态设置	指定初始播放动画	右键Set as Layer Default State	★★☆☆☆
动画融合机制	层级权重控制覆盖关系	通过Layer Weight参数调节	★★★★☆

操作步骤	功能说明	参数设置	预期效果
创建状态转移	连接两个动画状态	拖拽状态节点生成箭头	建立状态切换路径
设置float条件	数值比较触发切换	Greater/Less + 阈值	数值达标时激活转移
使用trigger	瞬时触发机制	无参数设置	单次触发后自动复位
多条件组合	复合逻辑判断	添加多个条件字段	需同时满足所有条件
层级权重调整	控制动画混合强度	Weight=1时完全覆盖	实现动画叠加效果

参数类型	比较运算符	适用场景	代码控制方式
Float	> <	连续数值变化(如速度)	SetFloat()
Int	> < = !=	离散状态(如技能等级)	SetInteger()
Bool	true/false	开关型条件(是否跳跃)	SetBool()
Trigger	无	瞬时动作 (受击反馈)	SetTrigger()